

Roadmap — Système de Gestion de Stock (SGS)

Stack : Spring Boot 3.4 / Java 17 / PostgreSQL — Angular

Méthode de travail : développement en tranches verticales. Chaque fonctionnalité = backend (entité/repository/service/controller/DTO) **puis immédiatement** le frontend Angular qui la consomme, avant de passer à la suivante

Sur l'ordre des fonctionnalités : ce n'est pas un ordre de priorité business, c'est l'ordre imposé par les dépendances du diagramme de classes (`Article` a besoin de `Categorie`, `Utilisateur` a besoin d'`Entreprise`, etc.). Une fonctionnalité ne peut pas être développée avant celles dont elle dépend — voir colonne "Dépend de".

Choix assumé : l'authentification arrive en fonctionnalité #11, pas en premier. Tout est non protégé jusque-là. C'est volontaire pour avancer vite en développement, pas un oubli

Liste des fonctionnalités, dans l'ordre de développement

#	Fonctionnalité	Dépend de	Statut
1	Catégories — CRUD complet	—	Backend ☒ · Frontend ☐
2	Articles — CRUD + lien catégorie, calcul TTC serveur	Catégories	Backend ☒ · Frontend ☐
3	Clients — CRUD (avec adresse embarquée)	—	☐
4	Fournisseurs — CRUD (avec adresse embarquée)	—	☐
5	Entreprise — infos entreprise (souvent un singleton ou géré par un admin)	—	☐
6	Utilisateurs & Rôles — gestion des comptes (sans sécurité active encore)	Entreprise	☐

#	Fonctionnalité	Dépend de	Statut
7	Commande Client — création commande + lignes, calcul du total	Clients, Articles	☐
8	Commande Fournisseur — création commande + lignes	Fournisseurs, Articles	☐
9	Réception fournisseur → Mouvement de stock (entrée)	Commande Fournisseur	☐
10	Vente — création vente + lignes, décrémentation stock, contrôle stock insuffisant	Clients, Articles, MvtStk	☐
11	Tableau de bord stock — niveaux, alertes rupture	Vente, MvtStk	☐
12	Authentification & autorisation — Spring Security + JWT, protection des routes	Utilisateurs & Rôles	☐
13	Finitions — validation globale, tests, nettoyage, Docker (optionnel)	Tout le reste	☐

Détail par fonctionnalité

1. Catégories ☒ backend fait

Objectif : classer les articles (ex : "Boissons", "Hygiène"...).

Backend (fait) :

- Champs `Categorie` : `id`, `designation`
- GET `/api/categories` → liste
- GET `/api/categories/{id}` → une catégorie
- POST `/api/categories` → création (DTO : `designation`)
- PUT `/api/categories/{id}` → modification
- DELETE `/api/categories/{id}` → suppression
- Règle : `designation` obligatoire, non vide

Frontend (à faire) :

- `CategorieService` (Angular `HttpClient`) : une méthode par endpoint
 - `Component` `categorie-list` : tableau + boutons éditer/supprimer
 - `Component` `categorie-form` : champ texte `designation`, réutilisé pour création et édition
 - `Route` `/categories`
-

2. Articles – backend fait

Objectif : gérer le catalogue produit avec prix et catégorie.

Backend (fait) :

- `Champs` `Article` : `id`, `codeArticle`, `designation`, `prixUnitaireHt`, `tauxTva`, `prixUnitaireTtc` (calculé serveur), `photo`, `categorie`
- `GET` `/api/articles`, `GET` `/api/articles/{id}`, `POST`, `PUT` `/api/articles/{id}`, `DELETE` `/api/articles/{id}`
- `Règle` : `codeArticle` unique, TTC recalculé côté serveur (jamais reçu du client), `categorieId` doit exister

Frontend (à faire) :

- `ArticleService`
 - `article-list` : tableau avec colonne catégorie, filtre déroulant par catégorie (appelle `CategorieService` pour peupler le filtre)
 - `article-form` : champs + `<select>` de catégories
 - `Route` `/articles`
-

3. Clients

Objectif : fiche client pour les commandes/ventes.

Backend :

- `Champs` `Client` proposés : `id`, `nom`, `telephone`, `email`, `adresse` (objet embarqué : `rue`, `ville`, `codePostal`, `pays`)
- `CRUD` standard (5 endpoints comme ci-dessus)
- `Règle` : `email` au format valide si renseigné, `nom` obligatoire

Frontend :

- `ClientService`
- `client-list`, `client-form` (le formulaire doit gérer les 4 sous-champs d'adresse comme un bloc)
- `Route` `/clients`

4. Fournisseurs

Objectif : fiche fournisseur pour les commandes d'achat.

Backend : identique à Clients (mêmes champs proposés : `id`, `nom`, `telephone`, `email`, `adresse`)

Frontend : identique à Clients, route `/fournisseurs`

5. Entreprise

Objectif : informations de l'entreprise qui utilise le logiciel (nom, coordonnées) – probablement un enregistrement unique en base (singleton), vu que ton diagramme la place à la racine du modèle.

Backend :

- Champs proposés : `id`, `nom`, `adresse`, `telephone`, `email`
- `GET /api/entreprise` (renvoie l'unique enregistrement, pas une liste), `PUT /api/entreprise` (mise à jour)
- Pas de `POST / DELETE` si on part sur un singleton – **à confirmer avec toi** : une seule entreprise gère l'appli, ou plusieurs (multi-tenant) ? Question posée plus haut dans notre échange, toujours en suspens.

Frontend :

- `entreprise-form` : un seul formulaire, pas de liste, route `/entreprise` (page admin)
-

6. Utilisateurs & Rôles

Objectif : comptes qui pourront se connecter (préparation de l'auth en #12).

Backend :

- Champs `Roles` proposés : `id`, `nom` (ex : ADMIN, VENDEUR, GESTIONNAIRE)
- Champs `Utilisateur` proposés : `id`, `nom`, `login`, `motDePasse` (hashé BCrypt dès maintenant, même sans sécurité active), `entreprise`, `roles` (liste)
- CRUD sur les deux, **sans protection d'endpoint pour l'instant** (ajoutée en #12)
- Règle : `login` unique, mot de passe jamais renvoyé dans les DTO de réponse

Frontend :

- `utilisateur-list`, `utilisateur-form` (avec sélection de rôle(s))

- Route `/utilisateurs` — accessible à tous pour l'instant, sera restreinte aux admins en #12
-

7. Commande Client

Objectif : un client passe commande de plusieurs articles en quantité, avant qu'elle soit transformée en vente.

Backend :

- Champs `CommandeClient` proposés : `id`, `dateCommande`, `statut` (`EN_COURS` / `VALIDEE` / `ANNULEE`), `client`, `lignes`
- Champs `LigneCommandeClient` proposés : `id`, `article`, `quantite`, `prixUnitaire` (copié depuis l'article au moment de la commande, pas recalculé après coup si le prix change), `commandeClient`
- `POST /api/commandes-client` : reçoit la commande **et ses lignes en une seule requête**, tout est sauvegardé dans une transaction (`@Transactional`) — soit tout réussit, soit rien n'est enregistré
- Endpoint qui renvoie le total calculé ($\text{somme } \text{quantite} * \text{prixUnitaire}$ des lignes)
- CRUD complémentaire : lecture, annulation

Frontend :

- `commande-client-form` : sélection client, ajout dynamique de lignes (choix article + quantité, bouton "+"), calcul du total affiché en direct côté client (recalculé aussi côté serveur, ne jamais faire confiance qu'au frontend)
 - `commande-client-list`
-

8. Commande Fournisseur

Objectif : symétrique de #7, côté achats.

Backend : même pattern que #7 avec `CommandeFournisseur` / `LigneCommandeFournisseur`

Frontend : même pattern que #7

9. Réception fournisseur → Mouvement de stock

Objectif : quand une commande fournisseur est physiquement reçue, le stock augmente.
(Section la plus susceptible d'évoluer une fois qu'on y sera.)

Backend :

- Champs `MvtStk` proposés : `id`, `article`, `typeMouvement` (ENTREE / SORTIE), `quantite`, `dateMouvement`, `origine` (référence vers la commande ou la vente à l'origine)
- POST `/api/commandes-fournisseur/{id}/receptionner` : marque la commande comme reçue, crée un `MvtStk` ENTREE pour chaque ligne, augmente le stock de chaque article
- Question à trancher avec toi à ce moment-là : le stock est-il un champ sur `Article` (`quantiteStock`) ou uniquement déduit en sommant les `MvtStk` ? Les deux se défendent, on en discute en #9.

Frontend :

- Bouton "Réceptionner" sur le détail d'une commande fournisseur
-

10. Vente

Objectif : cœur du métier — vendre des articles, décrémenter le stock, refuser si stock insuffisant.

Backend :

- Champs `Vente` proposés : `id`, `dateVente`, `client` (optionnel selon si vente au comptoir), `entreprise`, `lignes`
- Champs `LigneVente` proposés : `id`, `article`, `quantite`, `prixUnitaire`, `vente`
- POST `/api/ventes` : transaction qui crée la vente + lignes, génère un `MvtStk` SORTIE par ligne, décrémente le stock
- Règle métier stricte : si quantité demandée > stock disponible pour un article → toute la vente est rejetée (pas de vente partielle), erreur explicite renvoyée

Frontend :

- Écran de vente type "point de vente" : recherche article, ajout au panier, quantité, total, validation
 - Message d'erreur clair si stock insuffisant
-

11. Tableau de bord stock

Objectif : vue d'ensemble des niveaux de stock et alertes.

Backend :

- GET `/api/stock/etat` : liste des articles avec quantité en stock actuelle

- GET /api/stock/alertes : articles sous un seuil (seuil à définir — champ `seuilAlerte` à ajouter sur `Article`, ou valeur globale)

Frontend :

- Dashboard avec tableau/graphique simple des niveaux, mise en évidence des ruptures
-

12. Authentification & autorisation

Objectif : sécuriser l'API pour de vrai.

Backend :

- Activer Spring Security + JWT (déjà présent en commentaire dans `pom.xml`)
- POST /api/auth/login (login + mot de passe → JWT)
- Filtre de sécurité sur toutes les routes existantes, restrictions par rôle (ex : suppression réservée à ADMIN)
- **Passage en revue de toutes les fonctionnalités #1 à #11** pour vérifier qu'aucune route sensible n'est oubliée

Frontend :

- Page de login
 - Interceptor HTTP Angular qui attache le token JWT à chaque requête
 - Guards de route (redirection si non connecté / rôle insuffisant)
 - Affichage conditionnel des boutons selon le rôle (ex : bouton suppression caché si pas admin)
-

13. Finitions

- Validation systématique des DTOs restants
 - Tests unitaires sur `VenteService` et `CommandeClientService` (les plus critiques, calculs + règles métier)
 - Nettoyage `.gitignore`, README
 - (Optionnel selon temps dispo) Dockerisation backend + frontend + Postgres
-

Comment on utilise ce document

Une fonctionnalité = une session de travail complète (backend + frontend), pas plus. Tu me dis "je passe à la #3", je code, tu testes en Swagger d'abord (backend isolé) puis en Angular

(intégration), tu me rapportes ce qui casse, on corrige avant de passer à la suivante. Je continue à signaler les problèmes que je vois même si tu ne les as pas demandés (angles morts sécurité, règles métier manquantes, incohérences de design) — c'est le deal de départ.

Prochaine étape immédiate : #1 et #2 côté frontend Angular (les deux backends sont prêts), puis #3 Clients en entier.